

Federated Learning Project Report

Luo Xiaoyu - 50087474

1. Introduction

As the amount of data and requirement for training of deep neural networks continue to grow, the weaknesses of traditional centralized training approaches have also emerged—such as challenges with user data privacy and security, massive-scale training and communication inefficiency. To solve these weaknesses in centralized model including data privacy, massive data scale, and inefficient communication, federated learning (FL) has been raised as a potential solution. FL is a decentralized machine learning setting that completely changes the traditional model based on "centralized data, centralized training". Instead of collecting all the user data on one server, in federated learning it is left on users' devices (such as smartphones or tablets). By aggregating locally computed updates from these devices, a global model is iteratively learned. "Federated Learning is a machine learning setting where the goal is to train a high-quality centralized model with training data distributed over a very large number of clients each with unreliable and relatively slow network connections." [1]

This project leverages FL by implementing its core algorithm in more practical application scenarios. The primary target is to utilize the basic FedAvg algorithm and apply it to the Fashion-MNIST dataset. This project aims to extensively explore how this basic version FL algorithm trade off against simple centralized neural network baseline through different parameter settings.

2. System Design

2.1 Overall Architecture

The system consists of a central server and multiple clients taking part in a federated learning training round each epoch. The server initially broadcasts the global model to all clients using a set of ports. Once receiving this model, each client trains locally using this global model along with its corresponding local dataset. After training is finished, updated local models are sent back to the server to synchronize. The server aggregates these models using FedAvg algorithm to generate an updated global model, and then this global model gets broadcasted to clients for further iterations until convergence.

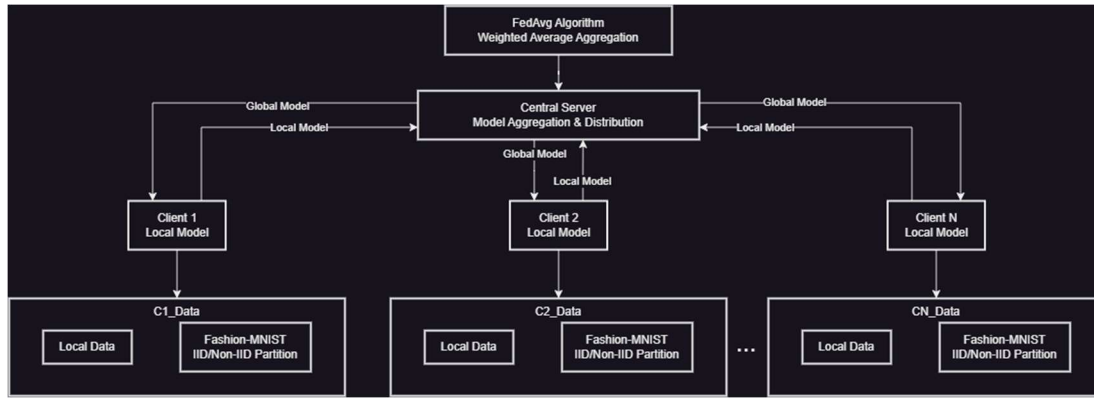


Figure 1: Overall Design of the Federated Learning model

The data flow in the system follows specific steps:

1. **Data Loading & Partitioning:** Load Fashion-MNIST dataset containing 60,000 training images and 10,000 test images into csv format. All images are divided according to the distribution mode IID(Independent and Identically Distributed) or Non-IID across N clients in order to form n mini datasets.
2. **Initialization:** Server initializes global model SimpleCNN and each client creates a local copy of it.
3. **Model Distribution (per round):** At the beginning of each communication round, server distributes current global model parameters to all participating clients for adjusting their local models accordingly.
4. **Local Training:** Each client trains its local model with n mini train dataset for 2 epochs which means raw dataset does not leave client's device.
5. **Parameter Upload:** Clients send their local updated parameters back to server along with information about their dataset size in order to update global weight accordingly.
6. **Model Aggregation:** The server collects all client parameters and applies the FedAvg algorithm—performing weighted averaging based on client data ratios—to update the global model.
7. **Model Evaluation:** The server evaluates the updated global model using the centralized test set, recording accuracy and loss metrics.
8. **Iteration:** Steps 3–7 repeat iteratively for 200 communication rounds until training completion.

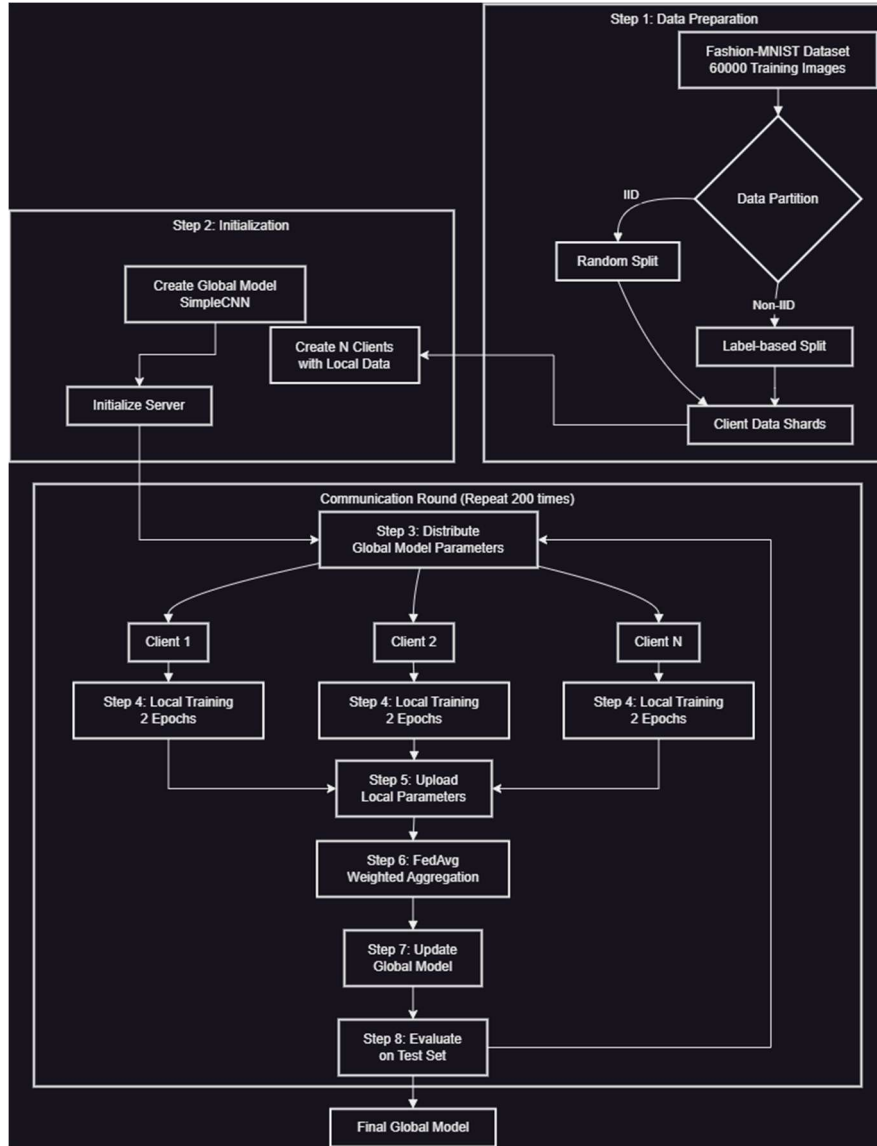


Figure 2: Data Flow of the project

2.2 Core Components

The data partition module represents the most basic function of splitting centralized data into client datasets. It takes the full Fashion-MNIST training dataset, containing 60,000 images with labeled data; and an N number of clients where it outputs a Python dictionary that maps a client ID to a local dataset. The functionality differs according to the distribution mode:

For IID, the module randomly shuffles and partitions the data evenly across each client.

For Non-IID, the module sorts the data based on their labels to have clusters of similar classes, divides the sorted data into $N \times 2$ fragments where each fragment includes only one or two dominant classes, and randomly assigns clients two different fragments which creates an intentional class imbalance.

The client module functions as a learning node that will train models locally without revealing sensitive training data. It will receive local dataset and global model parameters from the server then executes a local training round. Upon receiving two important inputs: assigned local dataset and current global model parameters from the server, it executes a training loop. It first updates its model parameters with global parameters received from the server, then performs forward pass operation on stochastic batch of its local dataset which computes prediction of classes, compute CrossEntropyLoss between predicted and ground truth labels, backward pass to compute gradient update for parameter optimization using Adam optimizer. This training process will continue for two complete epochs for its local dataset without sending raw data outside clients' device. Finally, it sends back two important output entities back to server: updated model parameters and size of its local dataset (to be used as weighting in averaging).

Server module acts as a managing entity in federated learning that will perform model aggregation and evaluation. It will receive clients updated model parameters and their respective datasets sizes. The key aggregate function is based on FedAvg algorithm which can be defined as:

$$W_{\text{global}} = \sum_{k=1}^K \frac{n_k}{n} \cdot W_k$$

Figure 3: Equation of FedAvg algorithm

Where:

W_{global} represents the new global model parameters.

W_k represents the model parameters from the k-th client.

n_k represents the data volume of the k-th client.

n represents the total data volume across all clients.

K represents the total number of clients.

Once this weighted average process is completed, server evaluates this averaged global model by computing prediction accuracy and CrossEntropyLoss through test dataset at this global model. After aggregation process, aggregated parameter updates are sent back to clients to initiate next iteration of training whereas evaluation metrics keep tabs on global convergence.

The neural network module processes Fashion-MNIST classification task with SimpleCNN architecture that transforms input image into categorical predictions in an iterative manner. The input image size is 28×28 grayscale image input and it computes a probability distribution over 10 clothing categories through following sequential operations:

Layer	Operations	Input Size	Output Size	Functionality
Conv1	Conv + BatchNorm + ReLU + MaxPool	1×28×28	32×14×14	Extract low-level features
Conv2	Conv + BatchNorm + ReLU + MaxPool	32×14×14	64×7×7	Capture mid-level features
Conv3	Conv + BatchNorm + ReLU + MaxPool	64×7×7	128×3×3	Learn high-level features
Flatten	Tensor reshaping	128×3×3	1152	Convert to 1D vector
FC1	Linear + ReLU + Dropout(0.4)	1152	256	Feature recombination
FC2	Linear + ReLU + Dropout(0.4)	256	128	Feature compression
FC3	Linear	128	10	Generate classification logits

Figure 4: SimpleCNN Model Architecture

2.3 Training Configuration

After extensive empirical validation, An optimal configuration was determined which is adapted as baseline for all further experiments. The federated learning is performed under fixed settings where 5, 10 or 20 clients cooperate through 200 communication rounds where every communication round has 2 local epochs per round. Adam optimizer was implemented to train model with learning rate set to 0.0008 and batch size of 32 samples. All clients participate in every round of training whereas their individual data partitions are distributed under different IID or Non-IID settings.

3. Experiments and Results

3.1 Experimental Setup

All experiments were conducted on the Fashion-MNIST dataset with 200 communication rounds (or epochs for centralized training). The federated learning experiments used FedAvg algorithm with 2 local epochs per round.

3.2 Experiment 1: Effect of Client Numbers (IID)

This experiment compared federated learning performance with 5, 10, and 20 clients under IID data distribution.

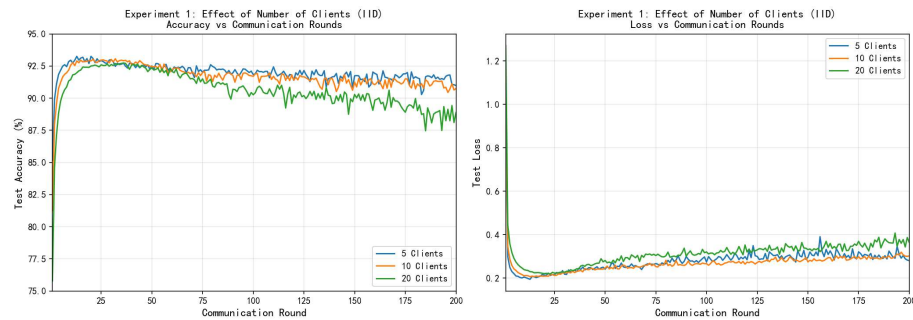


Figure 5: Experiment result of different client numbers

As seen in the Figure 5, all configurations gave similar accuracy (92-93%), but fewer clients led to faster convergence and more stable performance. 20-client setting had more variance due to less local data per client

3.3 Experiment 2: IID vs Non-IID Distribution

This experiment compared federated learning with 10 clients on IID and Non-IID data distribution.

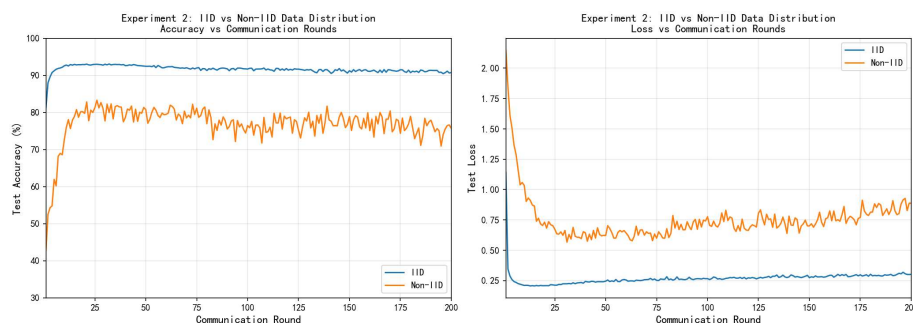


Figure 6: Experiment result of different data distribution methods

Non-IID setting had significant performance drop (15%) and high variance through the training. This represents challenge of statistical heterogeneity in federated learning. The issue is because every client's local data distribution differs significantly, so the model updates have diverged directions during aggregation.

3.4 Experiment 3: Centralized vs Federated Learning

This experiment compared centralized learning with federated learning (10 clients) under IID and Non-IID distribution of data. All models were trained for 200 epochs/rounds

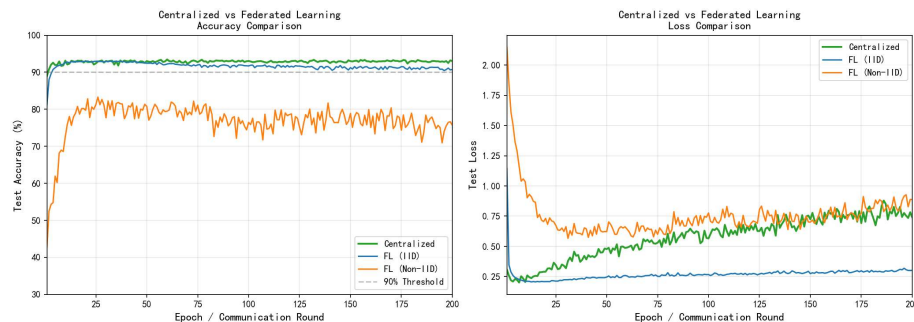


Figure 7: Experiment result of different learning model

FL(IID) model achieved peak accuracy similar to centralized model (93.07% v/s 93.46%) meeting the project goal successfully. FL(Non-IID) had significantly worse performance (~75%) due to heterogeneity in data. Also, FL had around twice training time compared to centralized learning due to communication overhead.

4. Challenges and Solutions

4.1 Non-IID Data Distribution

Challenge: As illustrated in experiment 2, Non-IID data distribution resulted in performance degradation (15%) and instability. Each client's local data has only few of total classes.

Solution: IID partitioning strategy was used in this project as base approach to ensure stable convergence. Future work will explore new advanced FL algorithms to make it robust and efficient when dealing with different distribution of data.

4.2 Trade-off Between Client Numbers and Performance

Challenge: If we increase number of clients, each client will have less data causing high variance in updates and slower convergence as seen in Experiment 1.

Solution: A balanced value of 10 clients was selected, providing sufficient parallelism while maintaining adequate data per client for stable training. Client fraction was set at 1.0 to ensure all clients participate in each round.

4.3 Communication Efficiency

Challenge: FL had around twice training time compared to centralized learning setup due to all the model transmission and aggregation overhead in each round of communication.

Solution: Number of local epochs was set at 2 allowing clients to do multiple local gradient updates before model averaging which reduced number of communication rounds required but maintaining same quality of model.

4.4 Hyperparameter Tuning

Challenge: Optimal hyperparameters like learning rate, batch size, number of local epochs is complex than centralized as it needs to give good performance across all clients.

Solution: Through empirical analysis, learning rate as 0.0008 using Adam optimizer and batch size as 32 was found to give good accuracy across different experiments configuration.

5. Conclusion

5.1 Advantages of Federated Learning

Federated learning offers several advantages over centralized training:

Data Privacy: Raw data is stored locally and never leaves a client's device, so only model updates are shared during training.

Distributed Computing: Training on local data reduces computation workload and communication overhead.

Scalability: The system can handle a large number of clients without requiring data to be centralized.

5.2 Experimental Results

This project developed a federated learning system using FedAvg on the Fashion-MNIST dataset. The results show:

FL achieves similar performance with the Centralized one under IID data distribution: the FL method reaches a peak accuracy of 93.07% and the Centralized version achieves 93.46%, satisfying the project requirement.

Number of Clients affect the stability of training: When the number of clients is around 5-10, it will have a more stable convergence results, comparing to using 20 clients; this is due to that when the client model has fewer local images, it converges less variably as it will have more similar local data

Non-IID data is hard: Since the model has achieved around 15% lower accuracy with variance in the non-IID situation, data distribution has an extreme impact on federated learning system.

Communication overhead increases training time: Due to the additional communication cost in each aggregation round, FL requires approximately 2x more training time compared to centralized learning.

5.3 Future Work

The primary limitation encountered in this project was implementing Federated Learning on Non-IID datasets. In Non-IID setting, a client downloads only a fraction of categories, causing specialization bias at every client where local models are unable to generalize well because they got trained on few classes only. So, solving this problem will be the future work.

In order to tackle this problem better algorithms need to be introduced. An algorithm named 'FedProx' addresses this issue and is an improved version of 'FedAvg'. "FedProx can be seen as a generalization and re-parametrization of FedAvg, the current state-of-the-art method for federated learning" [2].

FedProx modifies FedAvg by adding a proximal term in each client's local objective function which prevents it from varying too much from other clients' models during aggregation. This leads to more accurate averaging and even better convergence characteristics.

References

- [1] McMahan, B., Moore, E., Ramage, D., Hampson, S., & y Arcas, B. A. (2017, April). Communication-efficient learning of deep networks from decentralized data. In *Artificial intelligence and statistics* (pp. 1273-1282). PMLR.
- [2] Li, T., Sahu, A. K., Zaheer, M., Sanjabi, M., Talwalkar, A., & Smith, V. (2020). Federated optimization in heterogeneous networks. *Proceedings of Machine learning and systems*, 2, 429-450.